

Refactoring a microservice architecture

Chris Richardson

Founder of Eventuate.io

Founder of the original CloudFoundry.com

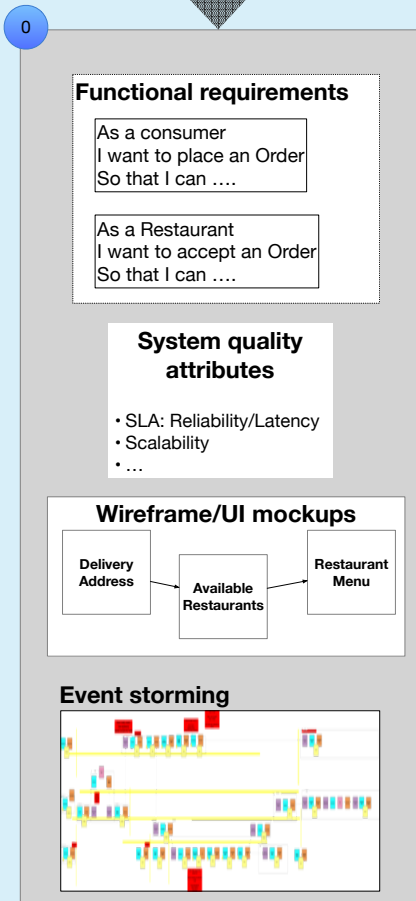
Author of POJOs in Action and Microservices Patterns

 @crichardson

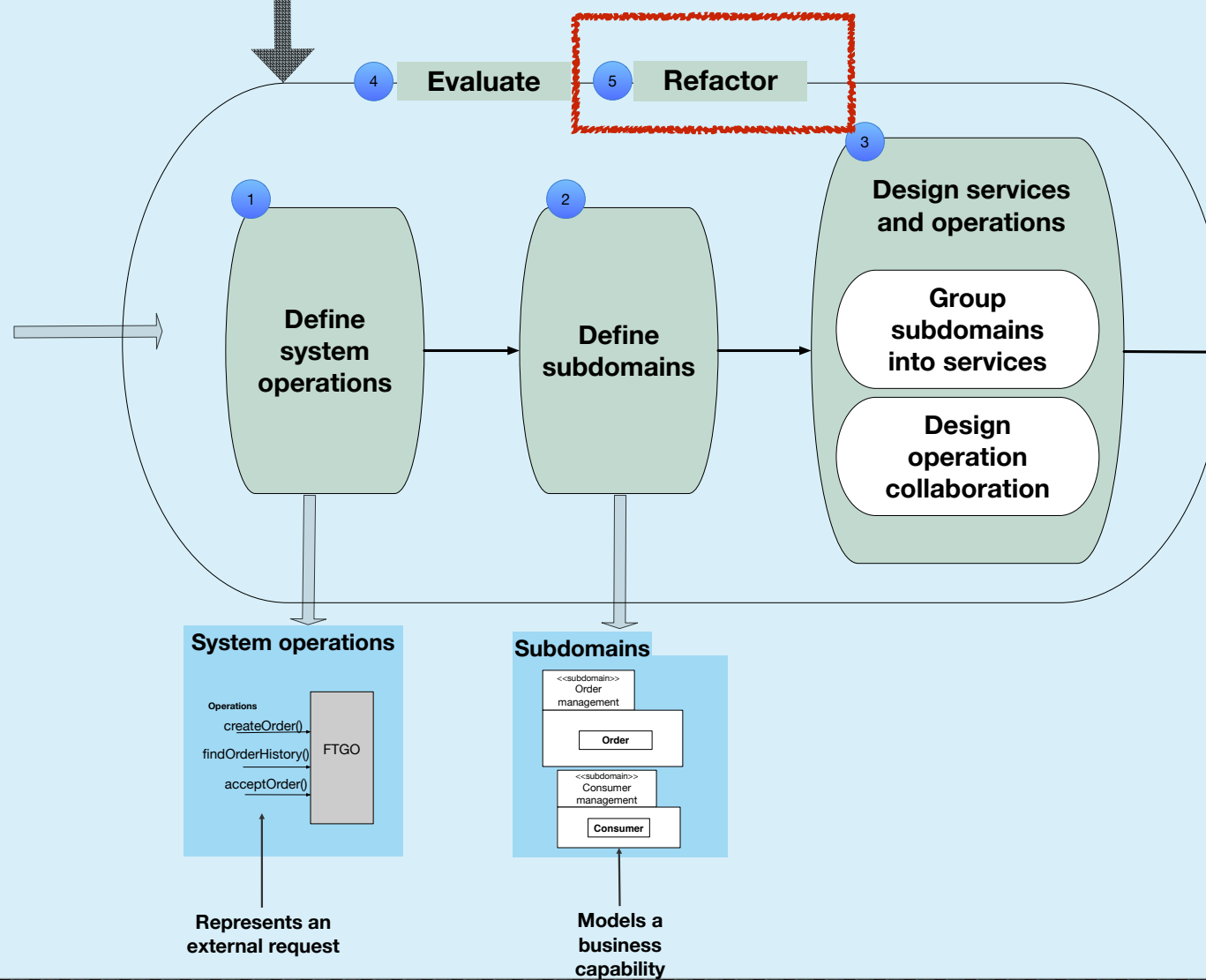
chris@chrisrichardson.net

adopt.microservices.io

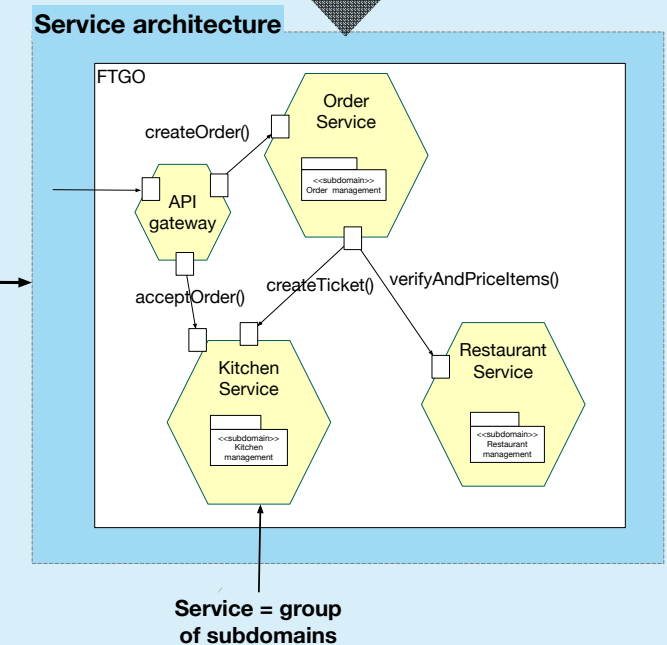
The starting point



The process



The result



Agenda

- ✦ Overview of refactoring an architecture
- ✦ Redesigning system operations
- ✦ Redesigning services by regrouping subdomains
- ✦ Redesigning subdomains and aggregates
- ✦ Redefining system operation specifications

Evaluate architecture:
Identify unresolved dark
energy and dark matter forces

=>

Refactor to improve
architecture

Revisit past design decisions:

- Redesigning system operations
- Redefining service responsibilities
 - Moving subdomains between services
 - Splitting and merging services
 - Refactoring subdomains and aggregates when needed
- Revising system operation specifications
 - To enable eventually consistency

**Refactor
architecture**

**Redesign
operations**

**Redefine service
responsibilities**

**Revise operation
specifications**

Identify problem

Brainstorm alternative designs that solve the problem
by altering past design decisions

Evaluate and compare designs using dark energy and
dark matter forces

Choose a design

Agenda

- Overview of refactoring an architecture
- Redesigning system operations
- Redesigning services by regrouping subdomains
- Redesigning subdomains and aggregates
- Redefining system operation specifications

Redesigning system operations

Change how a collaboration pattern is applied

OR

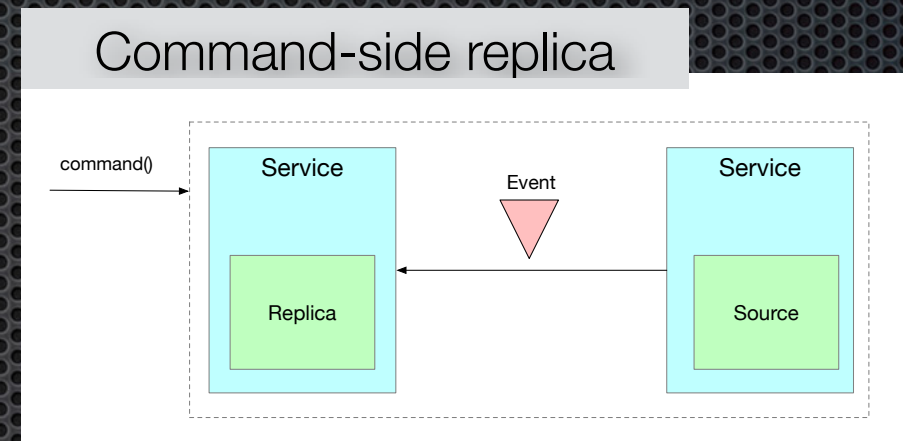
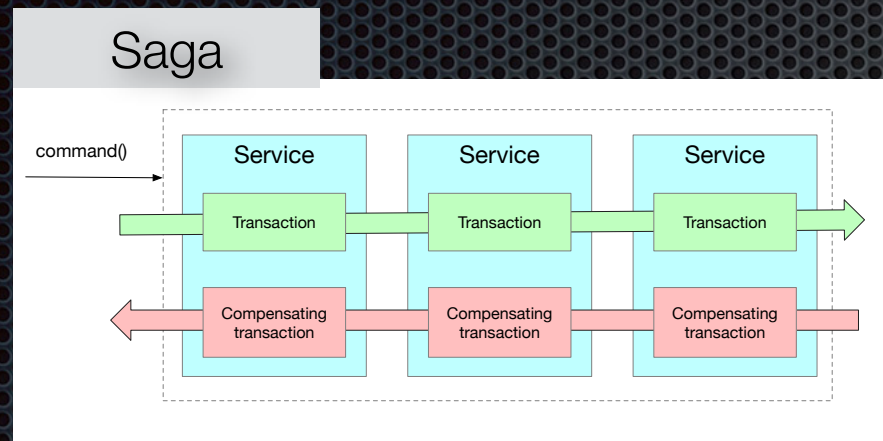
Switch collaboration patterns

Changing how a collaboration pattern is applied

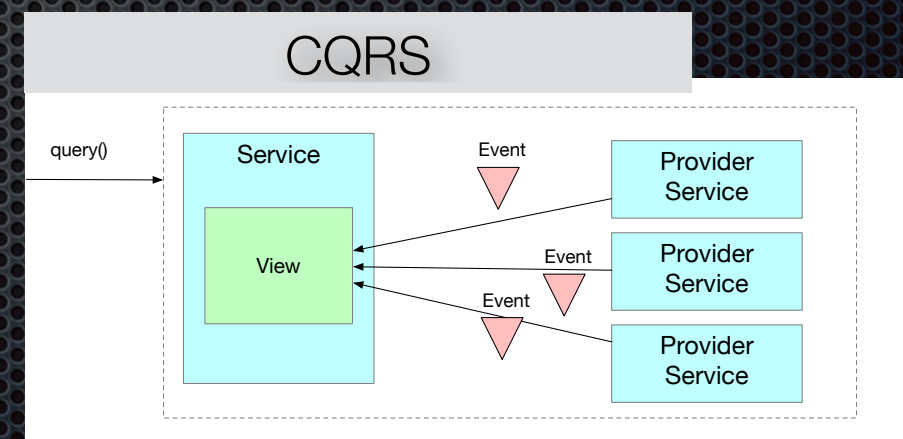
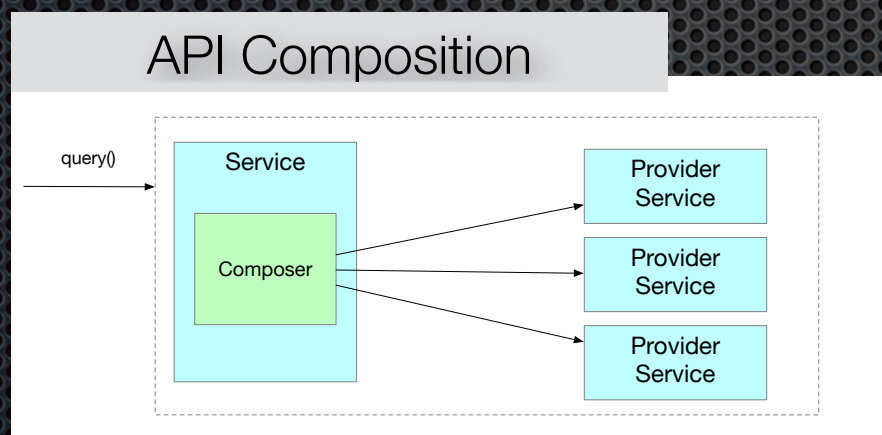
- ✦ Revisit the numerous design decisions that were made when applying the pattern
- ✦ For example:
 - ✦ Saga coordination: choreography vs. orchestration
 - ✦ Ordering of steps
 - ✦ ...

Switching patterns

Commands



Queries



Agenda

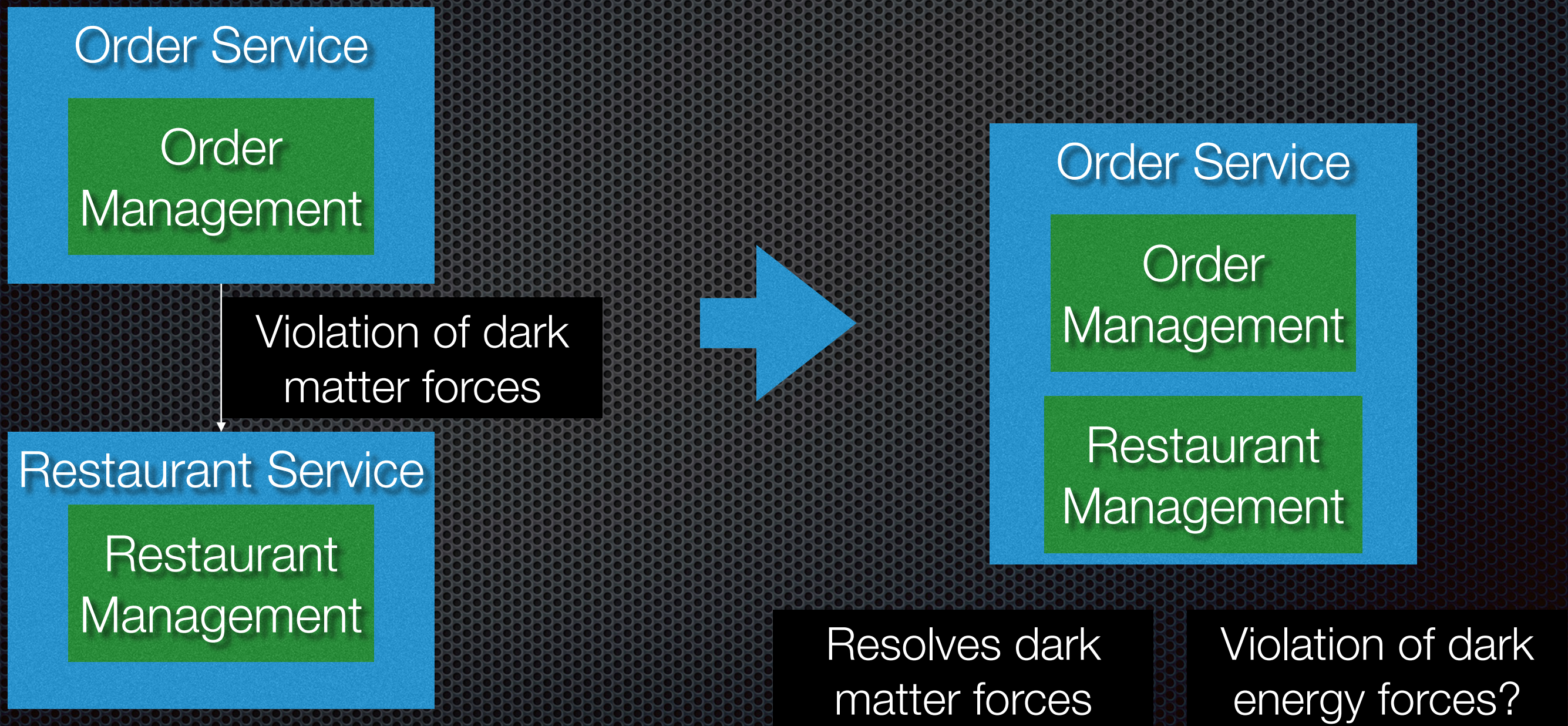
- ✦ Overview of refactoring an architecture
- ✦ Redesigning system operations
- ✦ Redesigning services by regrouping subdomains
- ✦ Redesigning subdomains and aggregates
- ✦ Redefining system operation specifications

Regrouping subdomains

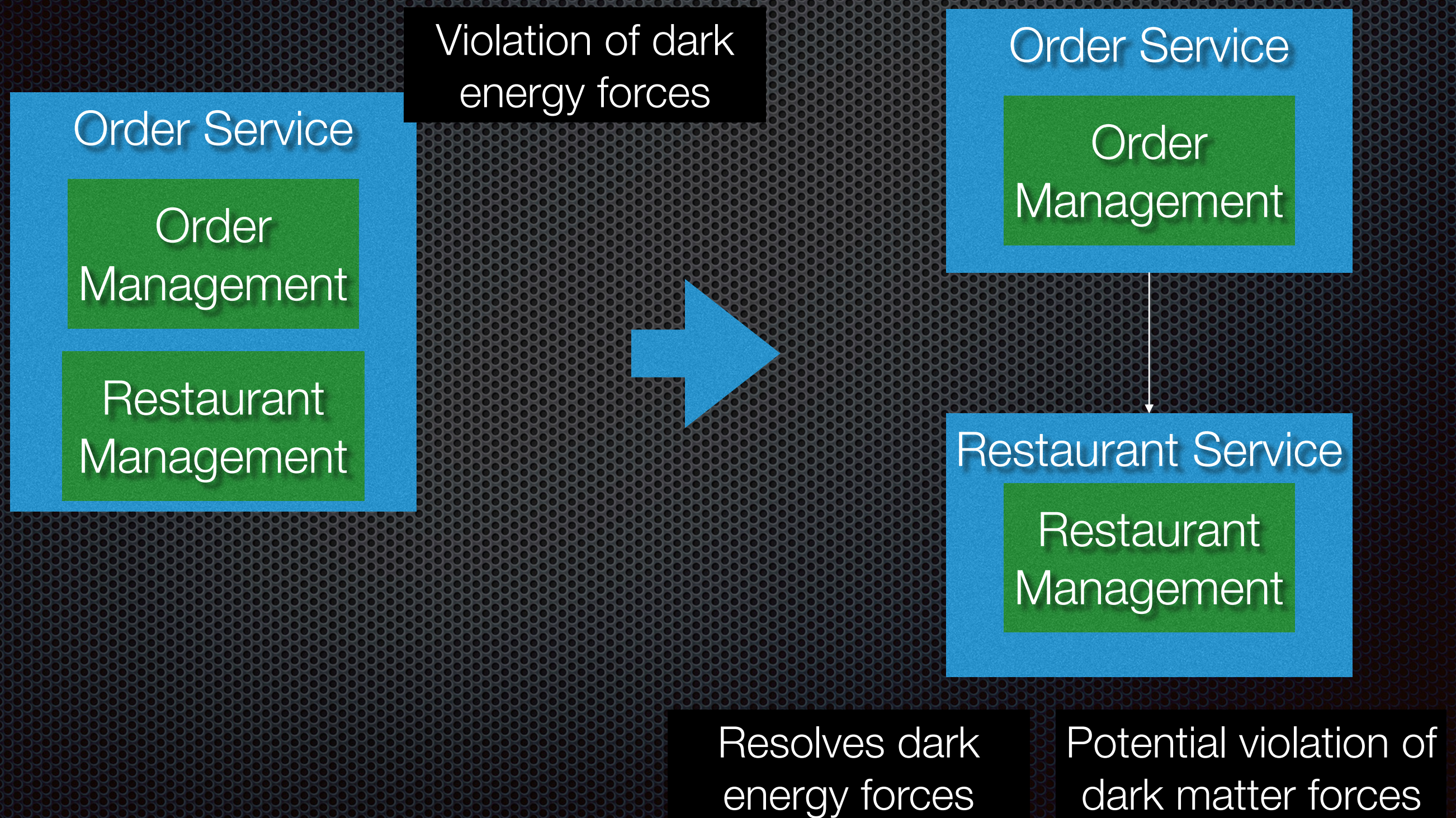
=

Changing assignment of
subdomains to services

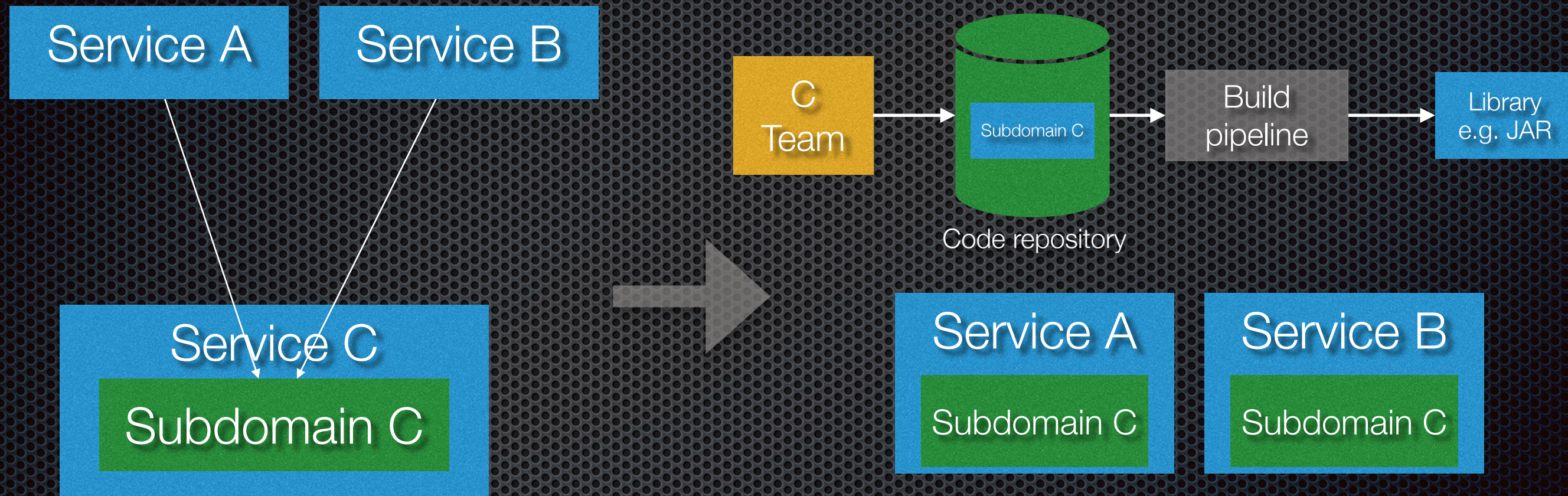
Example: merging Order and Restaurant Services



Example: splitting Order Service



Subdomain: Service => Shared library



Violation of runtime dark matter forces, e.g.

- Runtime coupling
- Performance

Potential violation of dark energy forces, e.g.

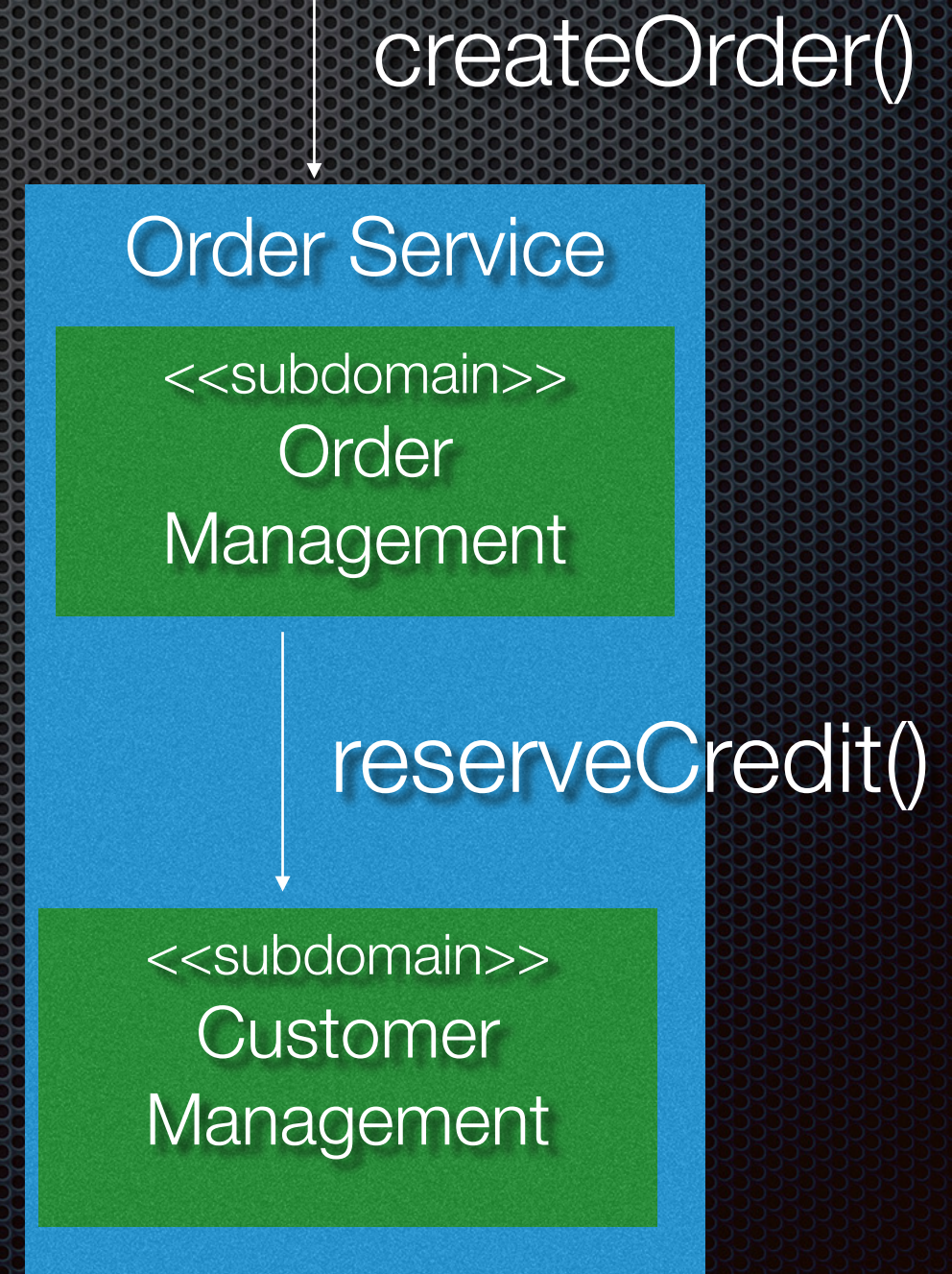
- Design-time coupling

Agenda

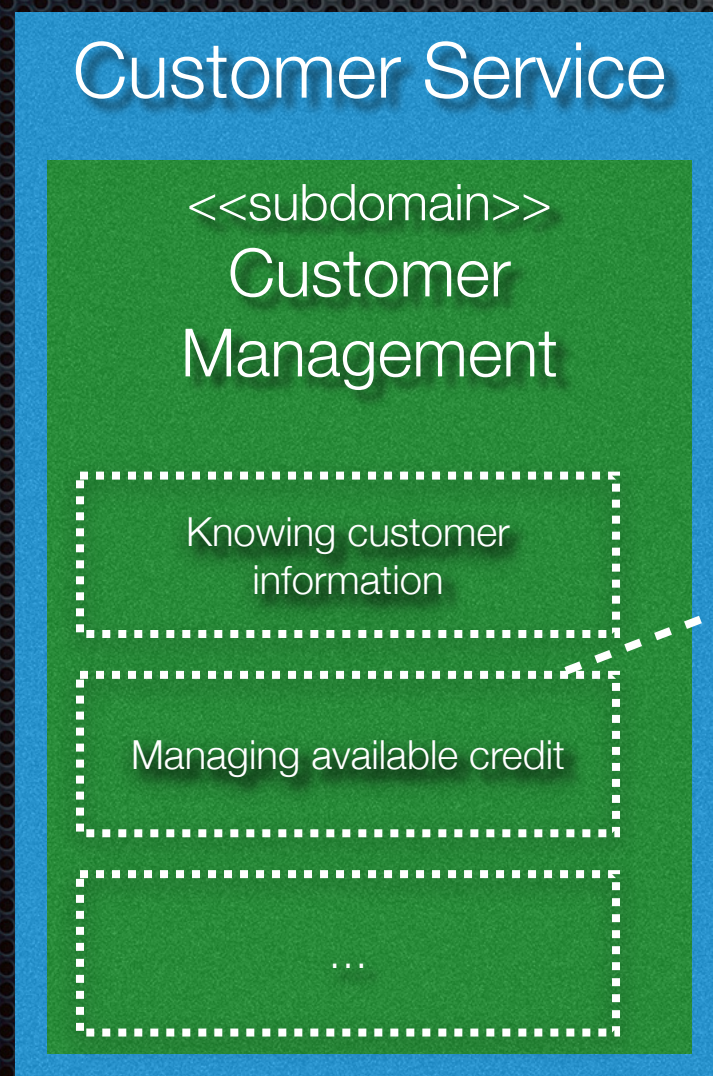
- Overview of refactoring an architecture
- Redesigning system operations
- Redesigning services by regrouping subdomains
- Redesigning subdomains and aggregates
- Redefining system operation specifications

Moving subdomains potentially = violates dark energy forces

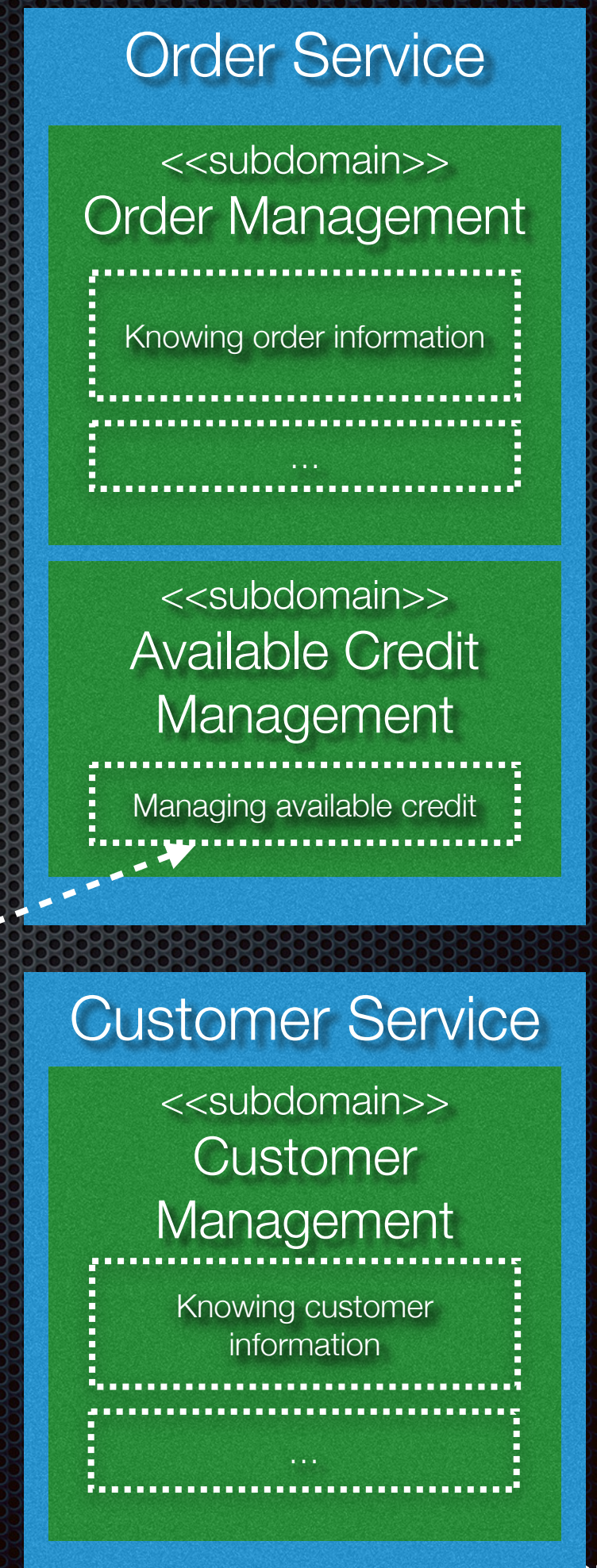
- Larger components
- Reduced team autonomy
- Slower deployment pipeline
- Single technology stacks
- Less segregation of domains
- ...



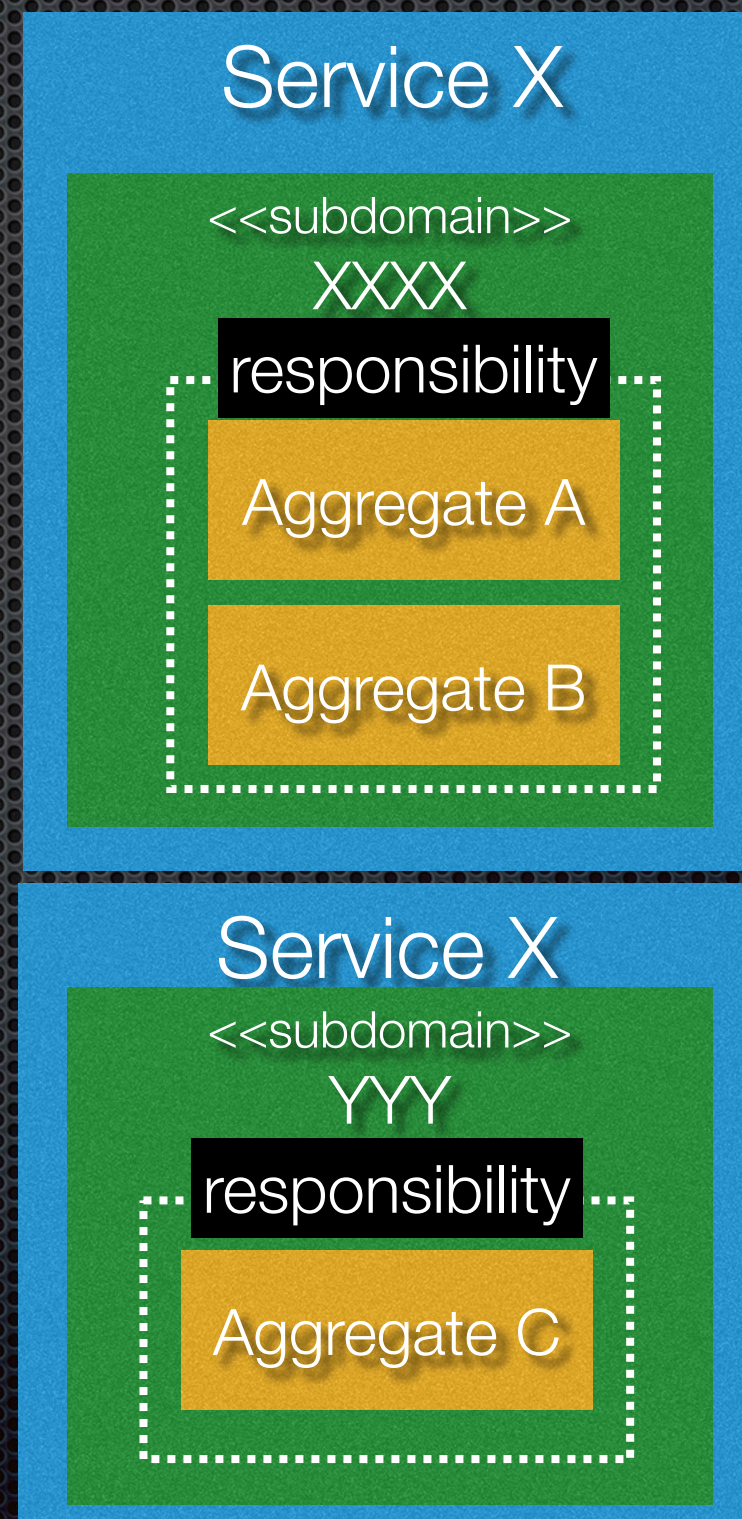
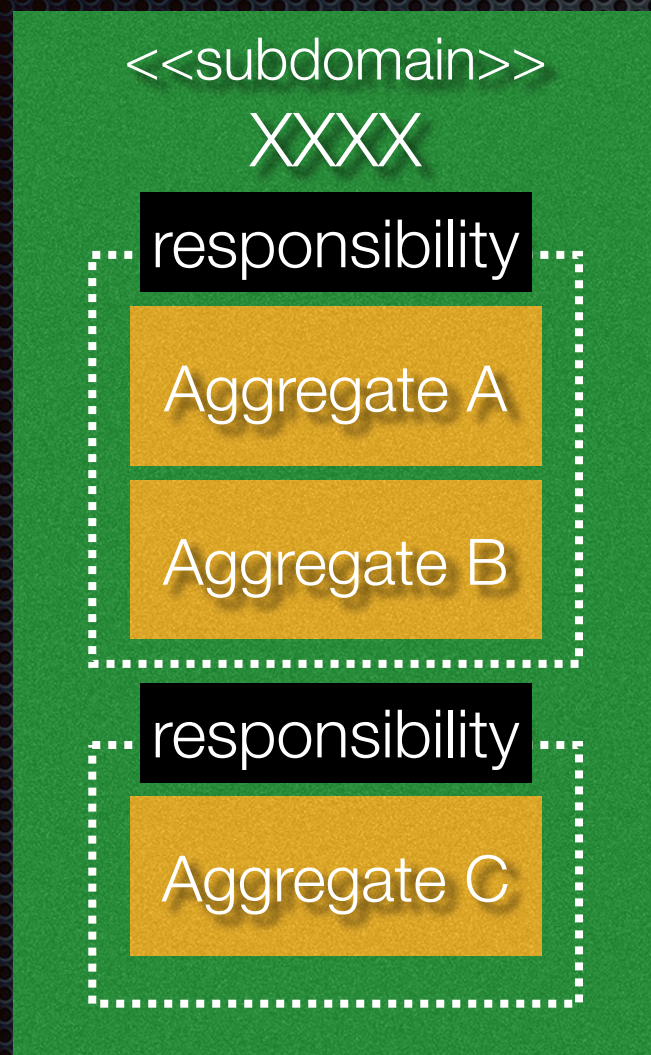
BUT Order management only needs **one responsibility**



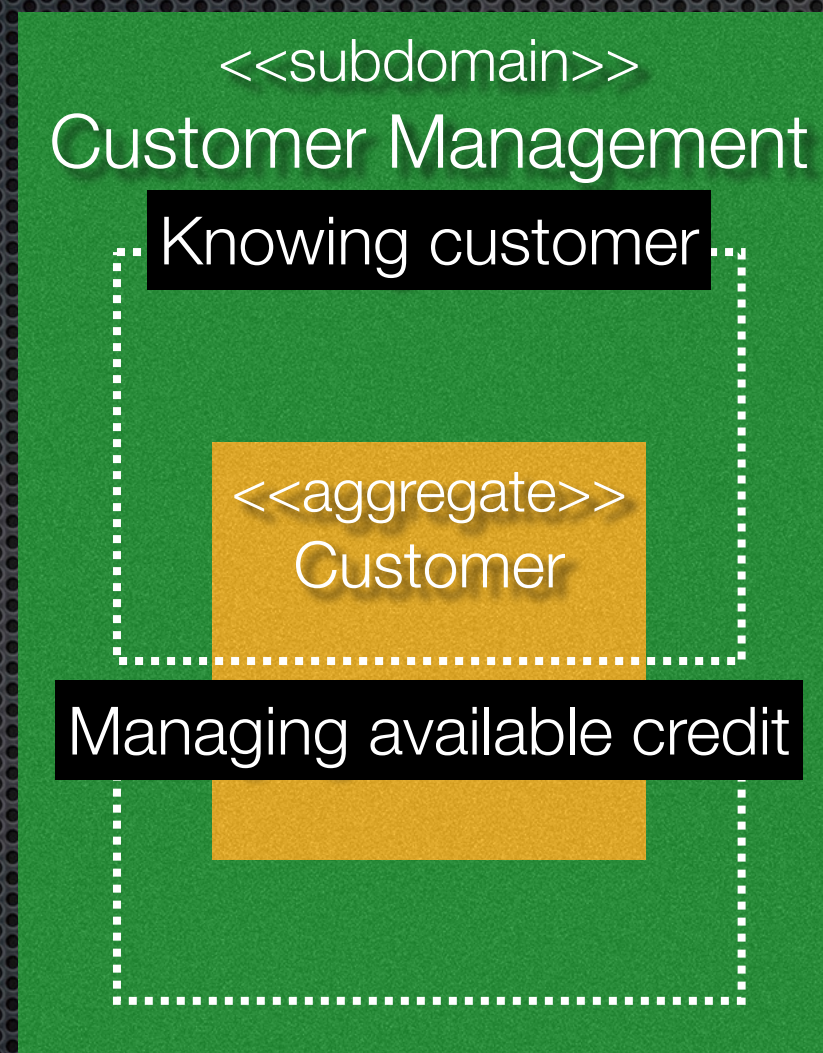
Move responsibility



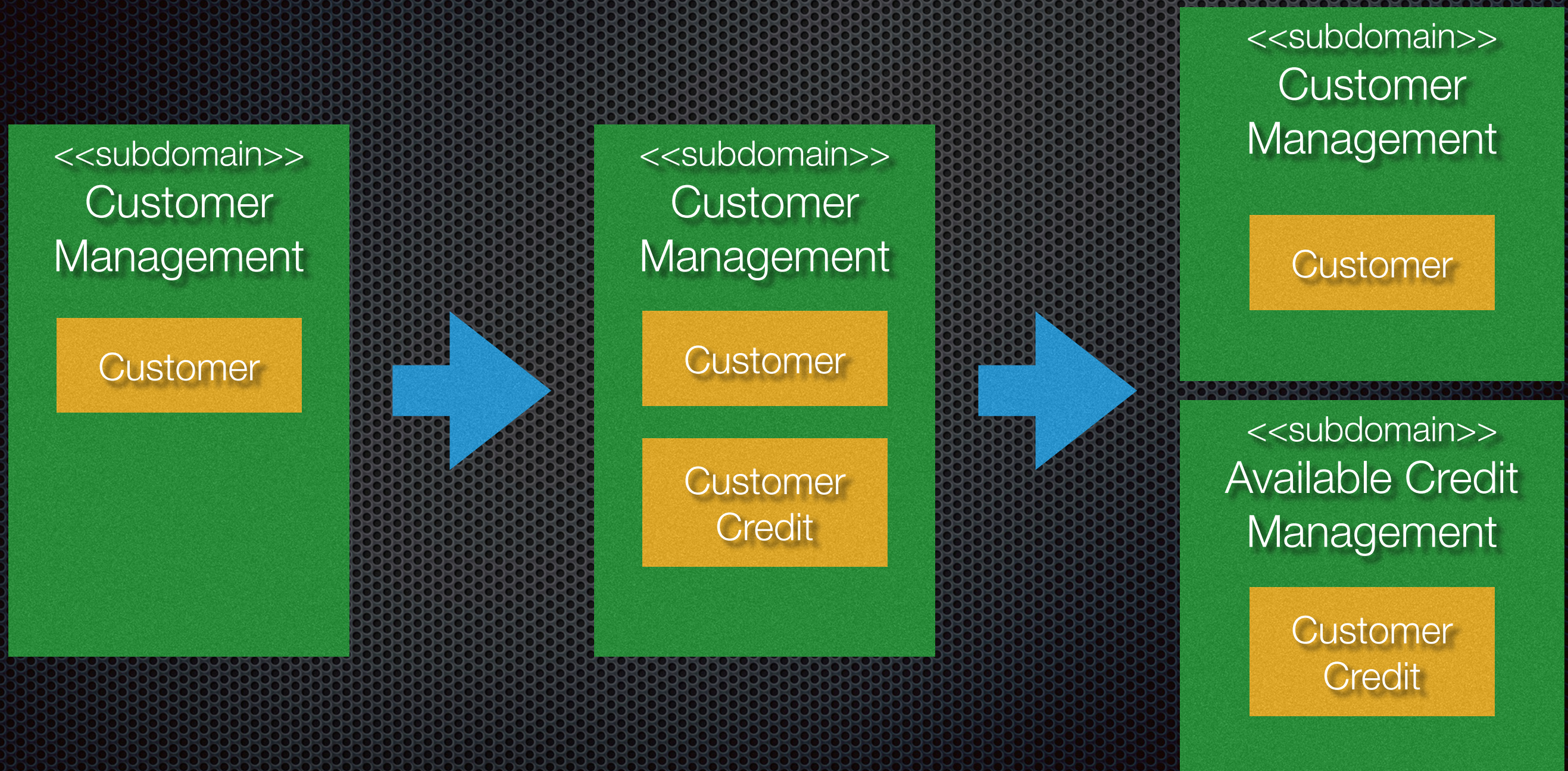
Refactoring subdomains: regrouping aggregates



But an aggregate can span responsibilities!



Refactoring subdomains: splitting aggregates



Refactoring “God” aggregates

Behavior but
no data!

Kitchen Service

Delivery Service

Accounting
Service

Coupling

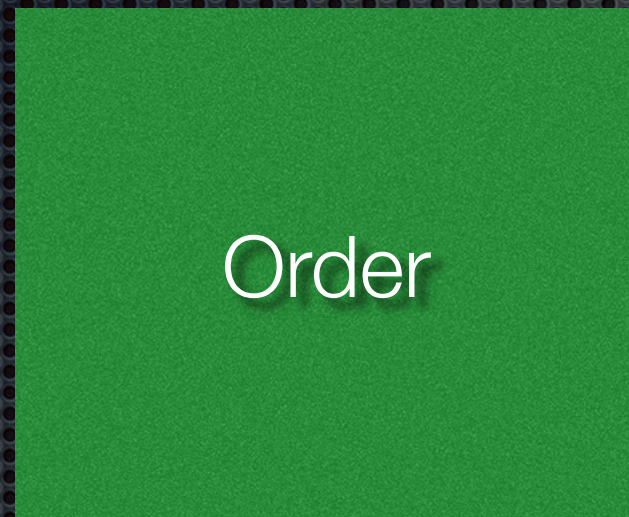
Order Service

Order

Referenced
Everywhere

Lots of data
+
Behavior

Refactoring Order



OrderService

Order

KitchenService

Ticket

DeliveryService

Delivery

AccountingService

Order

Agenda

- ✦ Overview of refactoring an architecture
- ✦ Redesigning system operations
- ✦ Redesigning services by regrouping subdomains
- ✦ Redesigning subdomains and aggregates
- ✦ Redefining system operation specifications

Operation specifications can resist decomposition

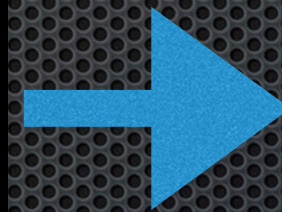
```
createOrder(customerId, orderTotal)
```

Pre-conditions:

- `orderTotal > 0`
- Exists Customer `c` where
 `c.ID = customerId`
 AND `c.availableCredit >= orderTotal`

Post-conditions:

- Order `o` was created
- `returnValue` became `o.ID`
- `o.total` became `orderTotal`
- `c.availableCredit -= orderTotal`



Response

Order
Management

Customer
Management

Potentially
violates dark
energy forces

Potentially
violates dark
matter forces

createOrder()

Order Service

Order
Management

Customer
Management

createOrder()

Order Service

Order
Management

runtime coupling

Customer
Service

Customer
Management

OR

Order
Management

Response

Customer
Management

Therefore: redefine operations to enable decomposition:

```
createOrder(customerId, orderTotal)
```

Pre-conditions:

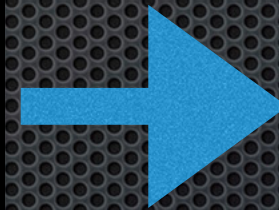
- `orderTotal > 0`
- **(E)** Exists Customer `c` where
 `c.ID = customerId`
 AND `c.availableCredit >= orderTotal`

Post-conditions:

- `Order o` was created
- `returnValue` became `o.ID`
- `o.total` became `orderTotal`
- **(E)** `c.availableCredit -= orderTotal`

(E) = eventually true

Before
response

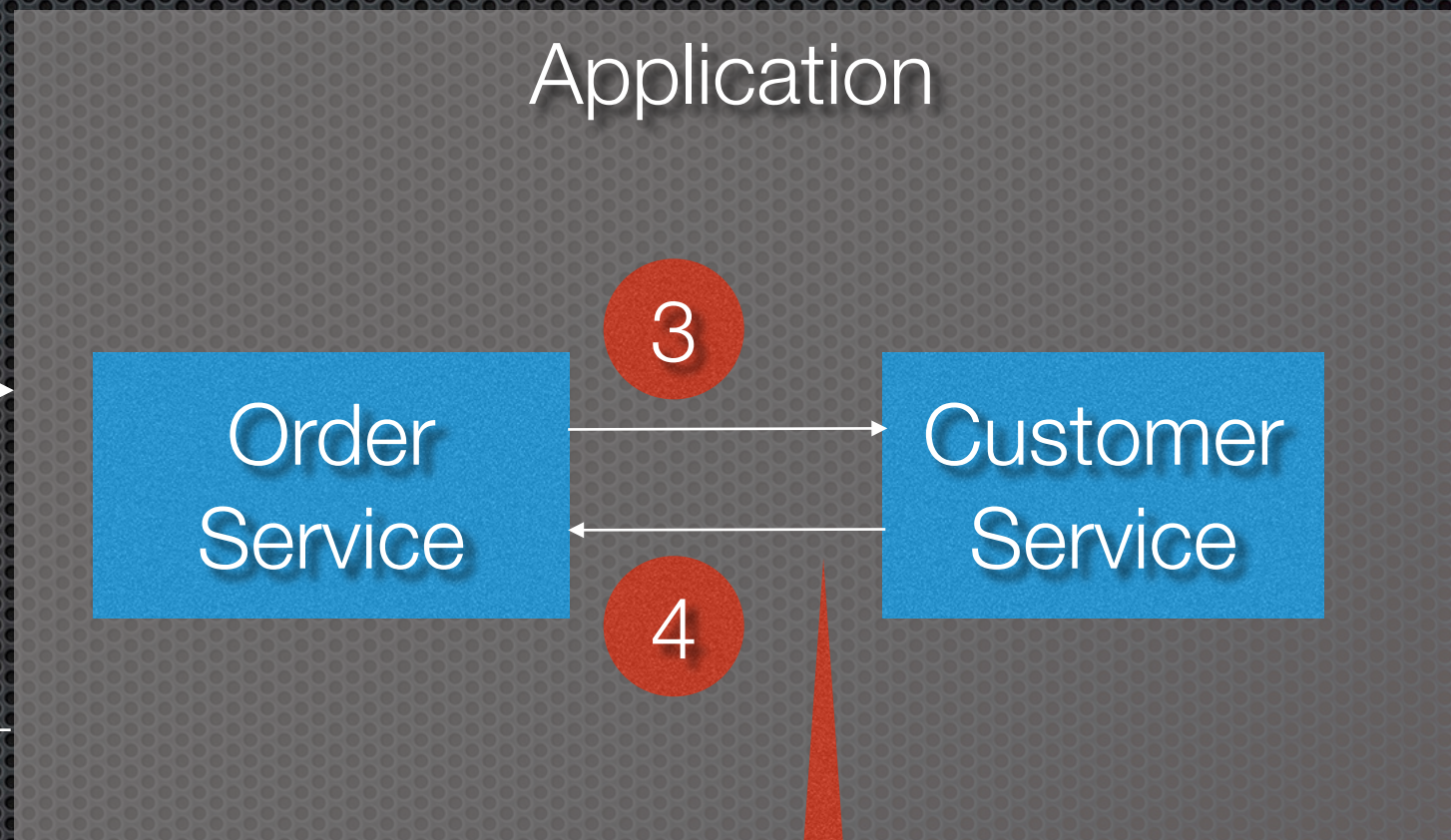
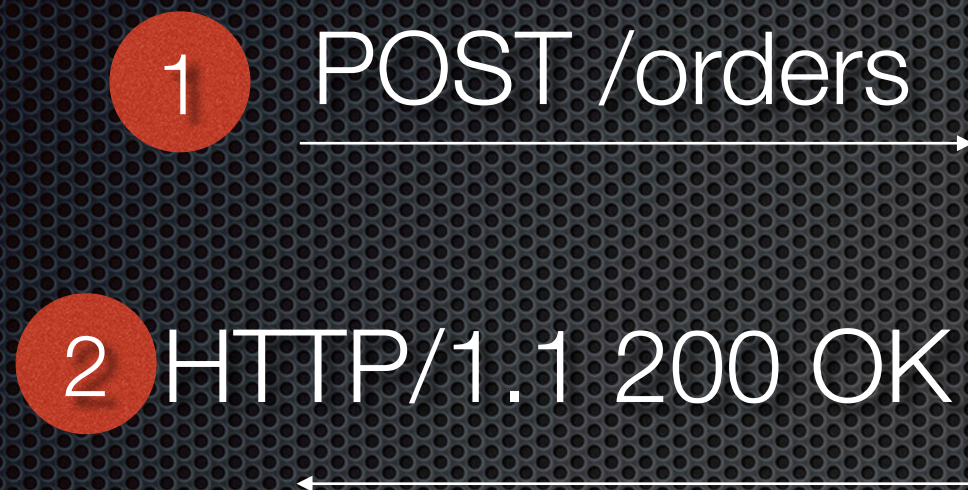


Response

Order
Management

After
response

Eventually = after response => supports decomposition



```
createOrder(customerId, orderTotal)
```

Pre-conditions:

- `orderTotal > 0`
- ~~Exists Customer c where~~
- ~~`c.ID == customerId`~~
- ~~`AND c.availableCredit >= orderTotal`~~

Post-conditions:

- Order o was created
- `returnValue` became `o.ID`
- `o.total` become `orderTotal`
- ~~`c.availableCredit -= orderTotal`~~

No runtime coupling
Remaining validations done later

Impact on API

- API changes:
 - createOrder() (HTTP POST /orders) returns partial outcome
 - Client must invoke getOrder() (HTTP GET /orders/{orderId})
- Requires negotiation with client team, e.g. UI developers

 [@crichardson](https://twitter.com/crichardson) chris@chrisrichardson.net

Questions?

adopt.microservices.io